LAB 1 - DELIVERABLES

Sequential Multiplier Using Repeated Addition

Name: R SARANG
Roll Number: EC23I2015
Course: VLSI Testing Practice (EC5034)

30th August 2026

Contents

Aim

To design and verify a 4-bit unsigned sequential multiplier using repeated addition, consisting of one adder and an FSM-based control path.

GitHub Repository: `https://github.com/rsarang14/VLSI-Testing-Practice`

1 Algorithm

The sequential multiplier computes the product of two 4-bit unsigned numbers (multiplicand A and multiplier B) using repeated addition. The algorithm initializes an 8-bit accumulator to zero. In each computation step, it adds the multiplicand to the accumulator and decrements the multiplier by 1. This process is repeated until the multiplier reaches zero.

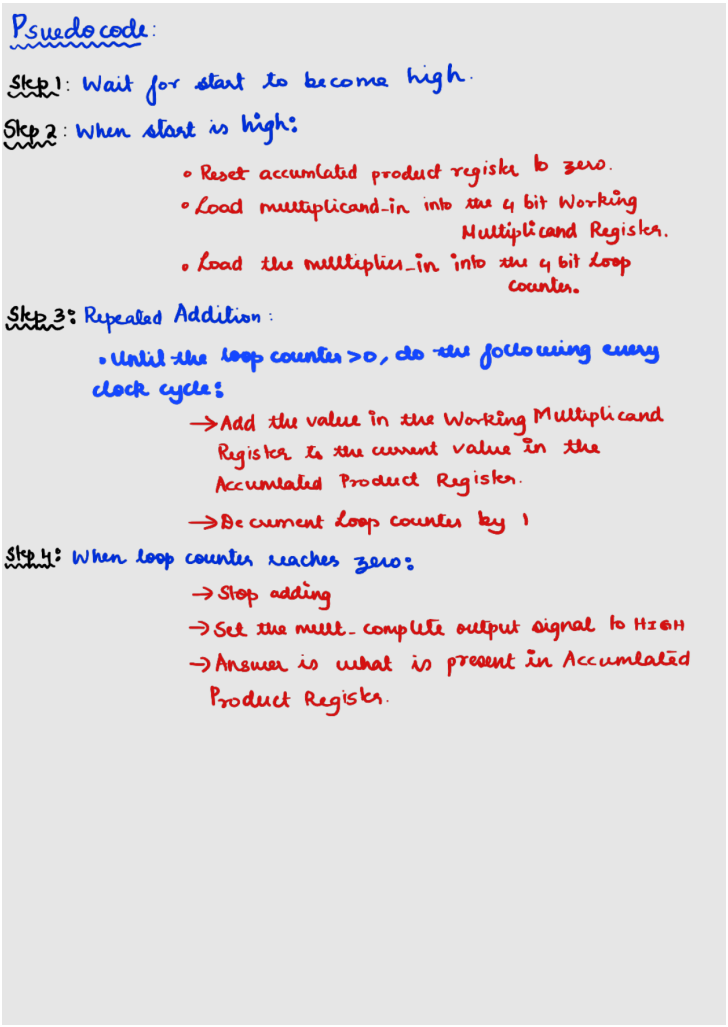


Figure 1: Algorithm Pseudocode

2 Datapath Elements

Datapath Element	Usage	Width
Working Multiplicand Register	Storing the i/p multiplicand_in.	4-bit
Loop Counter	Down counter that stores multiplier_in	4-bit
Accumulated Project Register	Storing the running sum & ultimately the required O/P.	8-bit
Adder	Adds the multiplicand to the accumulated product	8-bit
Zero Detector	Checking if the loop counter has reached zero	1-bit

Figure 2: Datapath Element Usage

- **Accumulated Product Register:** 8-bit register to hold the running sum.
- **Working Multiplicand Register:** 4-bit register to store operand A.
- **Loop Counter:** 4-bit down-counter to store operand B and decrement each cycle.
- **Adder:** 8-bit adder to add the working multiplicand to the accumulated product.
- **Zero Detector:** Combinational logic to check if the loop counter is zero.

3 Datapath Block Diagram

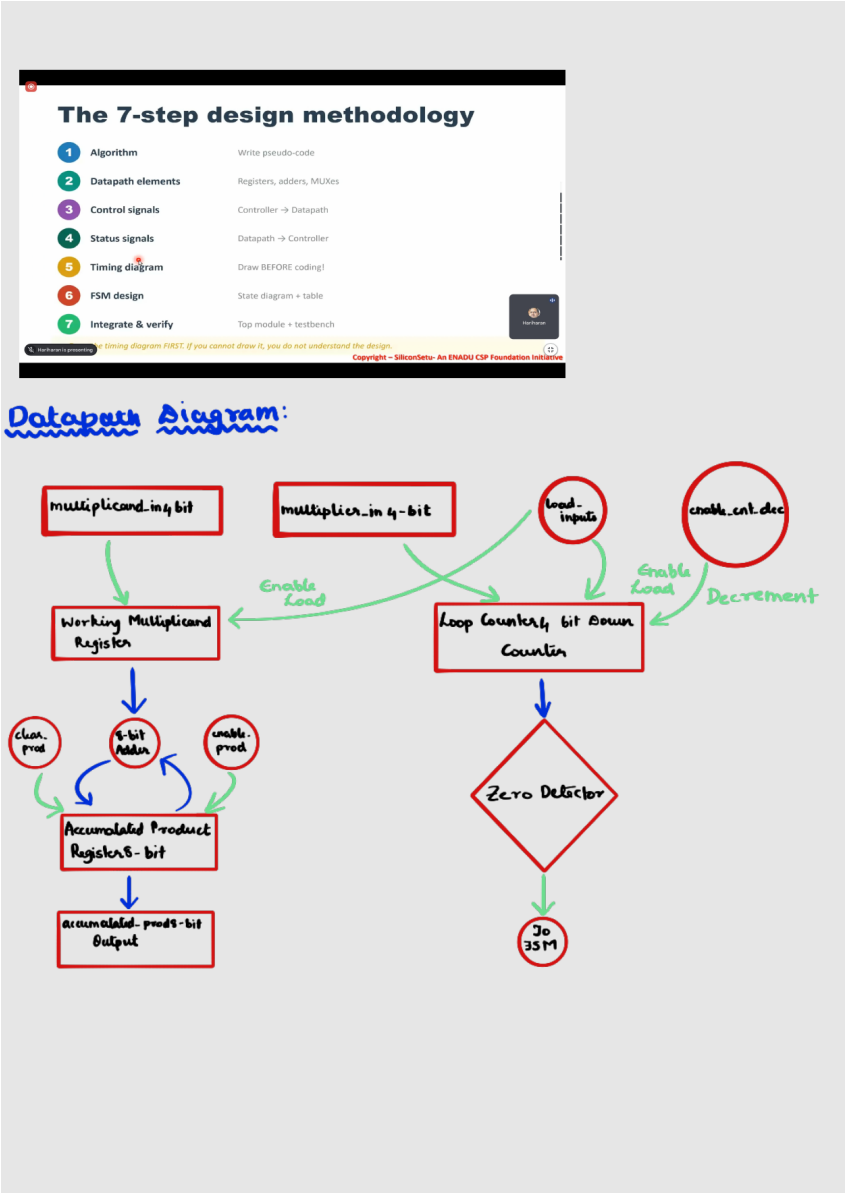


Figure 3: Complete Datapath Block Diagram

4 Control Signals

Control signals are generated by the FSM (Controller) and used to orchestrate the operations in the Datapath.

Signal	Direction	Function
`load_inputs`	Controller → Datapath	Loads the input operands (A into multiplicand reg, B into counter).
`clear_prod`	Controller → Datapath	Clears the 8-bit product register (accumulator) to 0.
`enable_prod`	Controller → Datapath	Enables product update (addition of multiplicand to accumulator).
`enable_cnt_dec`	Controller → Datapath	Enables loop counter operation (decrements the counter by 1).
`mult_complete`	Controller → Output	Indicates completion of the multiplication.

5 Status Signals

Status signals are generated by the Datapath and provided to the FSM (Controller) to make transition decisions.

Signal	Direction	Function
`loops_zero_flag`	Datapath → Controller	Indicates that the counter has reached zero.

6 Timing Diagram

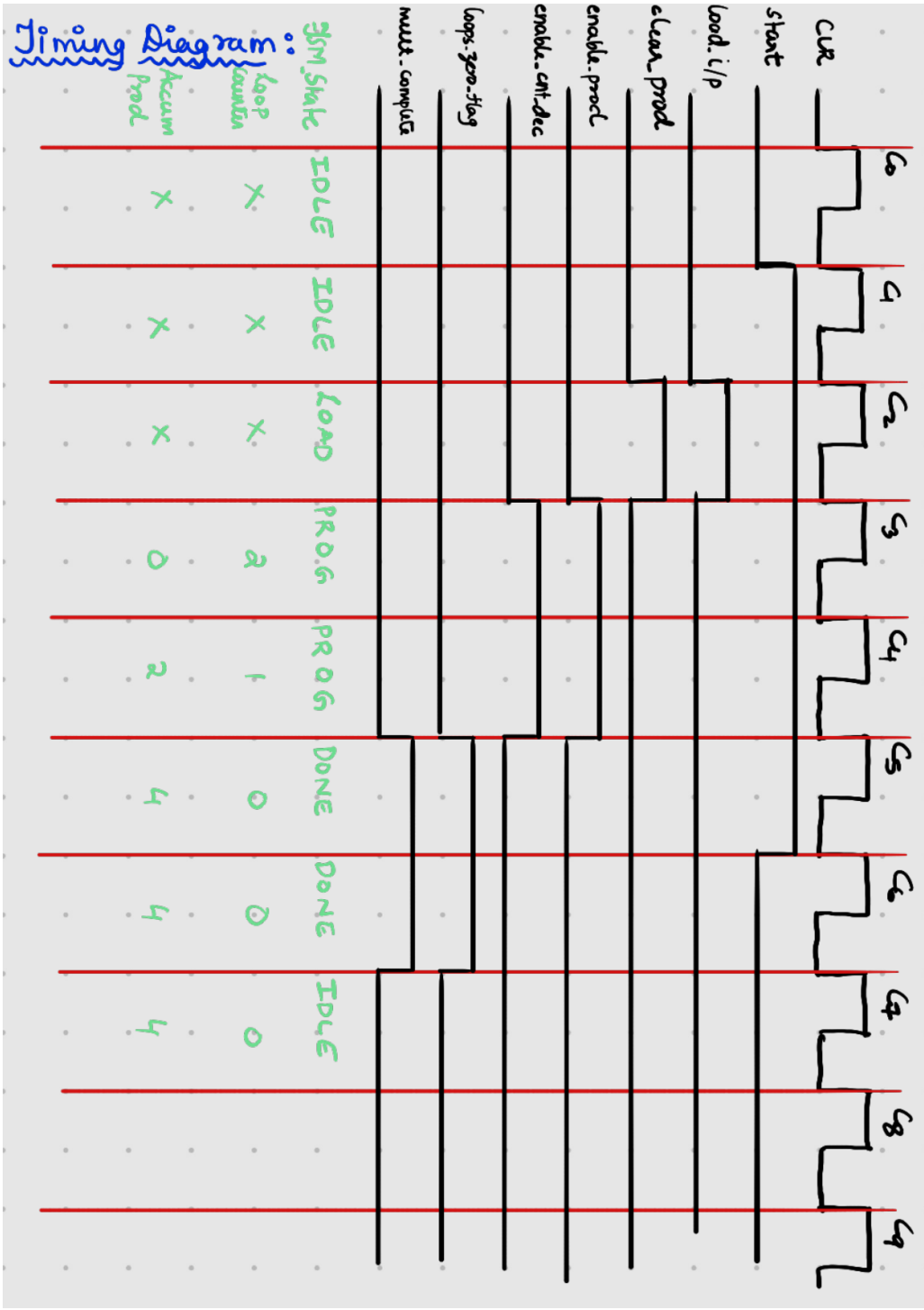
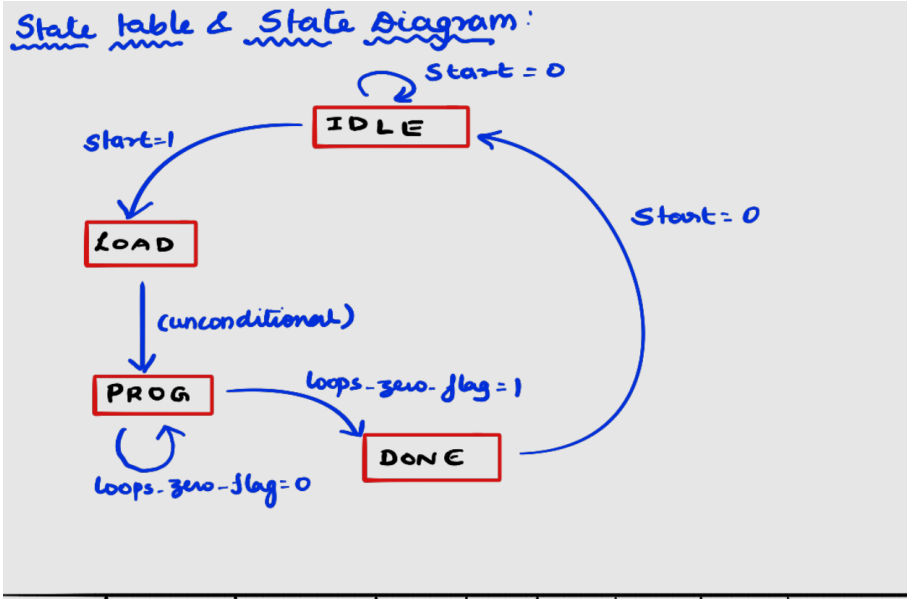


Figure 4: Timing Diagram for Control and Data Signals

7 FSM Design



Current State	start	loops_zero_flag	Next State	load_inputs	clear_prod	enable_prod	enable_cnt_dec	mult_complete
IDLE	0	X	LOAD	0	0	0	0	0
IDLE	1	X	LOAD	0	0	0	0	0
LOAD	X	X	PROG	1	1	0	0	0
PROG	X	0	PROG	0	0	1	1	0
PROG	X	1	DONE	0	0	0	0	0
DONE	1	X	DONE	0	0	0	0	1
DONE	0	X	IDLE	0	0	0	0	1

Figure 5: FSM State Table and State Diagram

Current State	start	loops_zero_flag	Next State	Outputs Asserted
IDLE (00)	0	X	IDLE (00)	None
IDLE (00)	1	X	LOAD (01)	None
LOAD (01)	X	X	PROG (10)	`load_inputs`=1, `clear_prod`=1
PROG (10)	X	0	PROG (10)	`enable_prod`=1, `enable_cnt_dec`=1
PROG (10)	X	1	DONE (11)	None
DONE (11)	1	X	DONE (11)	`mult_complete`=1
DONE (11)	0	X	IDLE (00)	`mult_complete`=1

8 Integrate & Verify

8.1 Verilog RTL

The Verilog RTL design consists of the datapath, controller, and top-level modules.

8.1.1 Datapath (`datapath.v`)

```
module datapath (
    input clk,

    // Inputs
    input [3:0] multiplicand_in,
    input [3:0] multiplier_in,

    // Control Signals
    input load_inputs,
    input clear_prod,
    input enable_prod,
    input enable_cnt_dec,

    // Status Signal
    output loops_zero_flag,

    // Data Output
    output reg [7:0] accumulated_prod
);

    // Internal Registers
    reg [3:0] working_multiplicand;
    reg [3:0] loop_counter;

    // APR (8-bit)
    // EC23I2015
    always @(posedge clk) begin
        if (clear_prod == 1'b1) begin
            accumulated_prod <= 8'd0;
        end
        else if (enable_prod == 1'b1) begin
            accumulated_prod <= accumulated_prod + working_multiplicand;
        end
    end

    // Working Multiplicand Register (4-bit)
    always @(posedge clk) begin
        if (load_inputs == 1'b1) begin
            working_multiplicand <= multiplicand_in;
        end
    end

    //4 bit Down Loop Counter
    always @(posedge clk) begin
        if (load_inputs == 1'b1) begin
            loop_counter <= multiplier_in;
        end
        else if (enable_cnt_dec == 1'b1) begin
            loop_counter <= loop_counter - 1; // Or loop_counter - 4'd1
```

```
        end
    end

    // Zero Detector
    assign loops_zero_flag = (loop_counter == 4'd0);

endmodule
```

8.1.2 Controller (`controller.v`)

```
module controller (
    input clk,
    input reset,

    // Going To FSM
    input start,
    input loops_zero_flag,

    // Coming from FSM (Control signals for datapath)
    output reg load_inputs,
    output reg clear_prod,
    output reg enable_prod,
    output reg enable_cnt_dec,

    // Output to outside world
    output reg mult_complete
);

    // Encodings for the required states
    parameter IDLE    = 2'b00;
    parameter LOAD    = 2'b01;
    parameter PROG    = 2'b10;
    parameter DONE    = 2'b11;

    // State Registers
    reg [1:0] current_state, next_state;

    // State Memory
    always @(posedge clk) begin

        if (reset ==1'b1) begin
            current_state <= IDLE;
        end
        else begin
            current_state <= next_state;
        end

    end

    // Next State & Output Logic
    // EC23I2015
    always @(*) begin
        // Default outputs
        load_inputs    = 1'b0;
        clear_prod     = 1'b0;
        enable_prod    = 1'b0;
        enable_cnt_dec = 1'b0;
        mult_complete  = 1'b0;
```

```
        next_state     = current_state;

        case (current_state)
            IDLE: begin

                if (start == 1'b1) begin
                    next_state = LOAD;
                end
            end

            LOAD: begin
                load_inputs = 1'b1;
                clear_prod = 1'b1;
                next_state = PROG;

            end

            PROG: begin
                if (loops_zero_flag == 1'b1) begin
                    next_state = DONE;
                    enable_prod = 1'b0;
                    enable_cnt_dec = 1'b0;
                end
                else begin
                    next_state = PROG;
                    enable_prod = 1'b1;
                    enable_cnt_dec = 1'b1;
                end
            end

            DONE: begin
                mult_complete = 1'b1;
                if (start==1'b1) begin
                    next_state=DONE;
                end
                else begin
                    next_state=IDLE;
                end
            end
        endcase
    end

endmodule
```

8.1.3 Top Module (`top.v`)

```
module top (
    input clk,
    input reset,
    input start,
    input [3:0] multiplicand_in,
    input [3:0] multiplier_in,

    output mult_complete,
    output [7:0] accumulated_prod
);

    // Internal Wires connecting Controller and Datapath
```

```
    wire load_inputs;
    wire clear_prod;
    wire enable_prod;
    wire enable_cnt_dec;
    wire loops_zero_flag;

    // Instantiation for the controller
    // EC23I2015
    controller u_controller (
        .clk             (clk),
        .reset           (reset),
        .start           (start),
        .loops_zero_flag (loops_zero_flag),
        .load_inputs     (load_inputs),
        .clear_prod      (clear_prod),
        .enable_prod     (enable_prod),
        .enable_cnt_dec  (enable_cnt_dec),
        .mult_complete   (mult_complete)
    );

    // Instantiation for the datapath
    datapath u_datapath (
        .clk              (clk),
        .multiplicand_in   (multiplicand_in),
        .multiplier_in     (multiplier_in),
        .load_inputs      (load_inputs),
        .clear_prod       (clear_prod),
        .enable_prod      (enable_prod),
        .enable_cnt_dec   (enable_cnt_dec),
        .loops_zero_flag  (loops_zero_flag),
        .accumulated_prod (accumulated_prod)
    );

endmodule
```

8.2 Testbench (tb_top.v)

```
`timescale 1ns / 1ps

module tb_top;

    // Testbench Inputs
    reg clk;
    reg reset;
    reg start;
    reg [3:0] multiplicand_in;
    reg [3:0] multiplier_in;

    // Testbench Outputs
    wire mult_complete;
    wire [7:0] accumulated_prod;

    // Instantiate Unit Under Test (UUT)
    top uut (
        .clk(clk),
        .reset(reset),
        .start(start),
        .multiplicand_in(multiplicand_in),
```

```
        .multiplier_in(multiplier_in),
        .mult_complete(mult_complete),
        .accumulated_prod(accumulated_prod)
    );

    // Clock Generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // 10ns Clock Period
    end

    // Test variables
    integer cycle_count;
    integer expected_product;
    integer expected_cycles;

    // Synchronous Test Task
    // EC23I2015
    task run_test(input [3:0] A, input [3:0] B);
        begin
            // Set inputs on falling edge of clock
            @(negedge clk);
            multiplicand_in = A;
            multiplier_in   = B;
            expected_product = A * B;
            expected_cycles  = B + 2; // B COMPUTE cycles + 1 LOAD + 1 DONE
            cycle_count      = 0;

            // Assert start signal
            start = 1'b1;

            // Wait 1 clock cycle for LOAD phase
            @(negedge clk);
            start = 1'b0; // De-assert start

            // Count cycles while waiting for completion
            while (!mult_complete) begin
                @(negedge clk);
                cycle_count = cycle_count + 1;
            end

            // Display self-checking results
            $write("Test: %2d x %2d | Exp Prod: %3d, Act Prod: %3d | Exp Cycles: %2d,
    ↪ Act Cycles: %2d | ",
                   A, B, expected_product, accumulated_prod, expected_cycles,
    ↪ cycle_count);

            if (accumulated_prod == expected_product && cycle_count == expected_cycles
    ↪ ) begin
                $display("[ PASS ]");
            end else begin
                $display("[ FAIL ]");
            end

            // Wait 2 clock cycles before starting next test (Synchronous delay)
            repeat (2) @(negedge clk);
        end
    endtask
```

```
77
78      // Test Execution Sequence
79      initial begin
80          // Initialize Inputs
81          reset = 1'b1;
82          start = 1'b0;
83          multiplicand_in = 4'd0;
84          multiplier_in   = 4'd0;
85
86          // Hold reset for 2 clock cycles
87          repeat (2) @(negedge clk);
88          reset = 1'b0;
89          repeat (2) @(negedge clk);
90
91          run_test(4'd0, 4'd0);
92          run_test(4'd0, 4'd5);
93          run_test(4'd5, 4'd0);
94          run_test(4'd1, 4'd1);
95          run_test(4'd1, 4'd15);
96          run_test(4'd15, 4'd1);
97
98          run_test(4'd3, 4'd4);
99          run_test(4'd15, 4'd15);
100
101         run_test(4'd3, 4'd12);
102         run_test(4'd12, 4'd3);
103
104         // Wait 2 clock cycles then finish simulation
105         repeat (2) @(negedge clk);
106         $finish;
107     end
108
109 endmodule
```

8.3 Test-Case Results

Test Case	A	B	Expected Product	Actual Product	Result
Corner (0x0)	0	0	0	0	PASS
Corner (0xN)	0	5	0	0	PASS
Corner (Nx0)	7	0	0	0	PASS
Corner (1x1)	1	1	1	1	PASS
Corner (1x15)	1	15	15	15	PASS
Corner (15x1)	15	1	15	15	PASS
Maximum Case	15	15	225	225	PASS
Commutativity 1	3	12	36	36	PASS
Commutativity 2	12	3	36	36	PASS

8.4 Cycle-Count Verification

Expected total cycles: $B + 2$ (Consists of 1 LOAD cycle + B COMPUTE cycles + 1 DONE cycle)

A	B	Expected Product	Expected Cycles	Actual Cycles	Result
0	0	0	2	2	PASS
3	4	12	6	6	PASS
3	12	36	14	14	PASS
12	3	36	5	5	PASS
15	15	225	17	17	PASS

8.5 3×12 vs 12×3 Observation

Both 3×12 and 12×3 result in the same product (36) due to the commutative property of multiplication. However, their execution times differ significantly in a sequential multiplier implemented using repeated addition.

- For 3×12 ($A = 3, B = 12$), the multiplier adds 3 to the accumulator twelve times. This requires 12 COMPUTE cycles, resulting in a total of 14 cycles ($12 + 2$).
- For 12×3 ($A = 12, B = 3$), the multiplier adds 12 to the accumulator three times. This requires only 3 COMPUTE cycles, resulting in a total of 5 cycles ($3 + 2$).

Conclusion: To minimize execution time in a repeated-addition sequential multiplier, the smaller operand should always be chosen as the multiplier (B).

Inference

The sequential multiplier was successfully designed by separating the logic into a datapath and a control path (FSM). The FSM effectively orchestrates the operations in the datapath, correctly managing the state transitions from IDLE to LOAD, to PROG (computation), and finally to DONE. Using a self-checking testbench, the design was validated across a comprehensive set of test cases including corner cases and worst-case scenarios, confirming its correctness and functional completeness.

Conclusion

In this lab, a 4-bit unsigned sequential multiplier was designed and verified using Verilog HDL. The separation of datapath and controller simplified the design process and debugging. Simulation results confirm that the FSM properly controls the repeated addition algorithm, producing accurate results for all test cases and corner conditions. The cycle-count analysis successfully demonstrated the performance dependency on the choice of the multiplier operand.